VPA Handle → Bank / PSP Mapping (for Stage 1's UPI Downtime API join)

Grounding: cross-checked across multiple UPI/VPA reference sources (Google Pay, PhonePe, Paytm official handle conventions). No single authoritative Razorpay-published list found — this is assembled from PSP-published handle conventions, not a Razorpay API response. Treat as a working reference to validate, not a doc-grounded artifact like the reason tables.

Why two categories exist

A VPA is `username@handle`. The `handle` always identifies the **PSP** (the app processing the payment). For some PSPs, the handle *also* encodes the specific bank — for others, it doesn't, because that PSP can sit on top of any bank account. This distinction directly determines whether a given transaction's `vpa` field can be joined against the Downtime API on a specific bank, or only against a specific PSP/route.

Category A — handle resolves directly to a bank code (usable for bank-level Downtime API join)

Handle	PSP (app)	Bank	Bank code (IFSC-style, same space as `card.issuer` / e-mandate `bank`)
`@okhdfcbank` / `@okhdfc`	Google Pay	HDFC Bank	`HDFC`
`@okaxis`	Google Pay	Axis Bank	`UTIB`
`@oksbi`	Google Pay	State Bank of India	`SBIN`
`@okicici`	Google Pay	ICICI Bank	`ICIC`
`@ptsbi`	Paytm	State Bank of India	`SBIN`
`@pthdfc`	Paytm	HDFC Bank	`HDFC`
`@ptaxis`	Paytm	Axis Bank	`UTIB`
`@ptyes`	Paytm	Yes Bank	`YESB`

Category B — handle resolves only to a PSP, not a specific bank (customer's underlying bank is unknown from the handle alone)

Handle	PSP (app)	Note
`@ybl`	PhonePe	Backend routes through Yes Bank, but the *customer's* linked account can be any bank — the handle names PhonePe's technical partner, not the customer's bank
`@axl`	PhonePe	Same caveat — Axis Bank is PhonePe's partner, not necessarily the customer's bank
`@ibl`	PhonePe	Same caveat — ICICI is PhonePe's partner here
`@paytm`	Paytm (generic/default handle)	No bank encoded
`@upi`	BHIM	No bank encoded
`@yapl`	Amazon Pay	No bank encoded
`@jio`	Jio Payments Bank	PSP-level only

What this means for Stage 1's join logic

Category A handles: extract the handle → look up the bank code → join directly against the Downtime API the same way E-mandate's `bank` field does. This is a real, bank-level correlation.
Category B handles: you can still cluster and correlate at the **PSP/route level** (e.g., "PhonePe transactions are failing at elevated rates in this window") — genuinely useful for detecting a PhonePe-side or NPCI-network-side issue — but you cannot claim a specific *bank* is down from the handle alone. If Stage 2 receives a Category B transaction and the cluster's `error_source` is `issuer_bank`, it's diagnosing a bank-side issue it can't independently confirm via this join — that's exactly the kind of case that should route toward `uncertain` rather than a confident `bank_outage`, unless corroborated some other way (e.g., cross-referencing against Category A transactions failing in the same window, if any exist in the batch).
This list is not exhaustive and not Razorpay-sourced. New PSPs/handles get onboarded by NPCI regularly. For your generator, this is fine as a fixed reference table (you control what handles you sample from), but if you present this in your architecture doc, be upfront that it's assembled from PSP conventions, not pulled from an official Razorpay endpoint the way the reason tables were — that's an honest distinction worth keeping in your Q&A back pocket.